

X68000 31 kHzモードで 60 Hz垂直同期を生成・実測する

MS.Xの実装を手がかりに、C実験とエミュレータ改善へつなげる

垂直タイミングの基礎・CRTC R04～R07・V-DISP実測・エミュレータ比較

MS.Xは60 Hzと約55 Hzの差を最初の壁として扱った

「MSXはNTSCモードで60Hz。ところが、X68000の通常の画面モードは約55Hz。このままでは垂直同期のタイミングがそもそも合いません。」[1, p.35]

「多くのゲームは『画面が書き換わる瞬間（垂直同期）』を基準に動いているので、ここがズレると、すべてが狂います。」[1, p.35]

1

MSXはNTSC系で約60 Hz

2

X68000の標準31 kHz系は約55.5 Hz

3

MS.XはCRTCを直接設定して差を解消した

C版では、この60 Hz化が本当に成立したかをV-DISPで実測する。

高性能なハードウェアでも、同じ時間動作は自動的に再現されない

「性能が高いこと」と「同じ振る舞いを再現できること」は別物

1

処理能力

CPUや描画機能に余裕があっても、同期周期の差は残る。

2

時間互換性

ゲームが参照する垂直同期を同じ周期で作る必要がある。

3

検証

設定値だけで判断せず、生成された信号を実時間で測る。

周波数の差

MSX NTSC : about 60 Hz
X68000 31 kHz : about 55.5 Hz

performance != timing compatibility

『MS.X開発秘話』PDF p.9 / p.37

この教訓を、525ラインを設定して600回のV-DISPを測る実験へ置き換える。

水平タイミングを保ったまま、垂直同期を約60 Hzへ変更する

MS.Xを参考に525ラインの垂直周期を作り、Cプログラムで実測する。

標準31 kHz系モードは約55.5 Hzで動作する。水平周波数を約31.5 kHzのまま維持し、1フレームを525ラインにすることで約60 Hzを目指す。

- 1 最初に映像信号の4区間とR04～R07の役割を理解する
- 2 次にMS.Xの設定を参考に、安全に復元できるC実験を作る
- 3 最後に実測結果から各エミュレータの課題と改善方向を整理する

設定した垂直同期が本当に60 Hzかを実測する。

MS.Xの実装からエミュレータ改善案までを5段階でたどる

値やコードを見る前に、垂直タイミングとレジスタの意味を確認する。



前半で生成と測定を説明し、後半でエミュレータが同じ動作をどう再現するか調査する。

1フレームは同期・空白・表示の4区間で構成される

バックポーチ：同期終了後～表示開始前

フロントポーチ：表示終了後～次の同期前

1

R04

1フレームの最終ラスタ。R04 + 1が総ライン数になる。

2

R05・R06

R05はVSYNC終端、R06はバックポーチ終端を表す。

3

R07

表示領域の終端。次のラスタからフロントポーチになる。

垂直タイミングの順序

VSYNC

- > back porch
- > visible area
- > front porch
- > next VSYNC

出典：XEiJ CRTC.java (R04～R07レジスタ定義)

R04～R07は区間の長さではなく、各区間が終わるラスタ位置を表す。

MS.XのソースでC版が操作するR04～R07を確認する

60 Hz化に必要な
垂直タイミングレジスタだけを
C版から操作する

1

R04

\$E80008。垂直総ライン数の終端位置を設定する。

2

R05・R06

\$E8000A / \$E8000C。同期と表示開始の境界を設定する。

3

R07

\$E8000E。表示領域の終端位置を設定する。

SRC/VDP/MS_VDP.H

```
#define CRTR_04 (*(volatile uint16_t*)0xe80008)
#define CRTR_05 (*(volatile uint16_t*)0xe8000a)
#define CRTR_06 (*(volatile uint16_t*)0xe8000c)
#define CRTR_07 (*(volatile uint16_t*)0xe8000e)
```

ms_vdp.h:28-37

MS.Xの定義から、C版で直接操作する4つのアドレスを確認できる。

31.5 kHzのまま525ラインにすると60 Hzになる

$$31,500 \text{ lines/s} \div 525 \text{ lines/frame} = 60.00 \text{ frames/s}$$

水平周波数

31.5 kHz

1ラインの時間は変更しない

垂直総ライン数

525 lines

4区間の合計

垂直周波数

60.00 Hz

1秒あたりのフレーム数

垂直周波数は、水平ライン周期と1フレームの総ライン数から決まる。

4区間の長さを累積してR04～R07の値を求める

VSYNC 2、バックポーチ33、表示480、フロントポーチ10を順に配置する。

VSYNC	バックポーチ	表示領域	フロントポーチ
2	33	480	10

$$2 + 33 + 480 + 10 = 525 \text{ lines}$$

R05=1 → R06=34 → R07=514 → R04=524

各レジスタには区間の最終位置を格納するため、総数より1小さい値になる。

MS.Xの設定をC版のR04～R07へ書き込む

垂直CRTC設定

```
R04 = 0x020C; /* 525 - 1 */  
R05 = 0x0001; /* VSYNC end */  
R06 = 0x0022; /* display start */  
R07 = 0x0202; /* display end */
```

MS.X / crtc60hz.c

MS.Xの垂直タイミング設定を参考に、C版でも525ラインの周期を作る。

1

標準モードから開始

IOCS mode 12で既知の31 kHz系水平タイミングを作る。

2

垂直値だけ変更

R04～R07へ導出した4つの値を書き込む。

3

生成後に実測

V-DISPを数え、設定結果を実時間で確認する。

MS.Xを設定値の根拠とし、C版は生成された垂直同期を測定する。

MS.Xを参考にし、C版は周波数測定と安全な復元を加える

比較項目	crtc60hz.c	MS.X
主な目的	垂直同期周波数を測る	MSX表示環境を構成する
CRTC設定	R04～R07だけを変更	表示モード全体を設定
周波数確認	600回のV-DISPを計測	1秒単位のtickを利用
終了時	保存した値を必ず復元	MS.Xの終了処理で復元

MS.Xの実装を参考にC版を作り、生成された垂直同期の周波数を実測する。

CRTC設定と画面モードを変更前に保存し、測定後に元へ戻す

CRTCを直接変更するため、復元までを1つの処理として設計する。



正常終了・ESC中止・タイムアウトのすべてで復元処理を実行する。

標準31 kHzモードを設定してから垂直値だけを変更する

処理の要点

```
_iocs_crtmod(12);  
  
save_crtc();  
write_crtc(R04, 0x020c);  
write_crtc(R05, 0x0001);  
write_crtc(R06, 0x0022);  
write_crtc(R07, 0x0202);
```

crtc60hz.c

水平タイミングを再設計せず、垂直周期だけを変更する。

1

先に標準モード

IOCSで既知の31 kHz系状態を作る。

2

元の値を保存

直接書き換える前にR04～R07を読む。

3

変更範囲を限定

垂直タイミング以外へ触れない。

危険なハードウェア操作を保存・設定・復元の3処理へ分ける。

GPIP bit 4を監視して次のV-DISP立ち上がりを待つ

現在の信号状態に関係なく、次の開始点を1回だけ検出する。

1

GPIP bit 4

General Purpose Input/Output Port。
\$10はV-DISP (0: 空白、1: 表示)。\$40はCIRQ。

2

一度Lowを待つ

呼び出し時がHighでも同じフレームを数えない。

3

タイムアウト

信号が変化しない場合は復元処理へ進む。

```
WAIT_VDISP()
```

```
#define GPIP_VDISP 0x10
```

```
while ((read_gpip() & GPIP_VDISP) != 0) { }  
while ((read_gpip() & GPIP_VDISP) == 0) { }
```

```
crtc60hz.c
```

1回のwait_vdisp()が、次のV-DISP開始を1回数える。

600回目のV-DISP直後に終了時刻を取得する

測定区間

```
wait_vdisp();
start = _iocs_ontime();

for (frame = 1; frame <= 600; ++frame) {
    wait_vdisp();
    if (frame == 600) end = _iocs_ontime();
}
```

crtc60hz.c

開始点と終了点をV-DISPの直後にそろえる。

1

開始

最初のV-DISP直後にstartを取得する。

2

600区間

その後のV-DISPを600回待つ。

3

終了

描画処理より前にendを取得する。

$$\text{Hz} \times 100 = 600 \times 10000 \div \text{経過時間 (1/100秒)}$$

描画を減らし、どの終了経路でも画面を元へ戻す

フローラインは測定を見やすくするための補助表示であり、測定そのものではない。

描画は約30回/秒へ抑え、終了前にラインを消去する。カーソル状態、CRTC R04～R07、IOCS画面モードは保存した順序に対応して復元する。

- 1 測定を妨げないよう描画処理を減らす
- 2 終了時にフローラインとグラフィック面を消去する
- 3 正常終了・ESC・タイムアウトで同じ復元処理を使う

```
X68000 60 Hz V-DISP measurement
Measuring 600 frames...
Progress   : 360 / 600
FPS : 60.00

ESC : abort
```

正常動作例：V-DISPを計数しながらFPS表示と1pxフローラインを更新

実機・XEiJ・XM6 TypeGでは約60 Hzを確認できた

同じプログラムでも、エミュレータによって測定結果が異なる。

実機

約60 Hz

525ライン設定と復元を確認

XEiJ / XM6 TypeG

約60 Hz

V-DISP周期と終了復帰を確認

WebX68k / MPX68k

不一致

時間管理の実装を調べる必要がある

差が生じる場所は、CプログラムではなくCRTC値を時間へ変換する処理と考えられる。

CRTC値から実時間まで4段階を正しくつなぐ

V-DISPのビットが存在するだけでは、正しい周波数にはならない。

1. CRTC設定

R04～R07から総ライン数と表示区間を求める



2. ラスタイベント

1ラインの時間に従ってラスタ番号を進める



3. MFP GPIF

表示開始・終了位置でbit 4を変化させる



4. 実時間同期

エミュレーション時間をホストの経過時間へ合わせる

この4段階をXEiJとXM6 TypeGで確認し、改善条件として使う。

XEIJはラスト時刻に従ってV-DISPを変化させる

XEIJ/CRTC.JAVA

```
crtVDispStart = crtR06VBackEndCurr + 1;  
crtVIdleStart = crtR07VDispEndCurr + 1;
```

```
MC68901.mfpVdispRise();
```

```
TickerQueue.tkqAdd(  
    crtTicker = NormalDrawDispSync,  
    crtClock += crtHFrontLength);
```

CRTC.java

画面描画回数ではなく、CRTCから計算したイベント時刻を使う。

1

表示区間を計算

R06+1とR07+1をイベント境界にする。

2

MFPへ通知

表示開始時にmfpVdispRise()を呼ぶ。

3

時刻を加算

水平区間の長さをcrtClockへ積み上げる。

CRTCの信号生成とホスト画面の表示を分けている。

XM6 TypeGでもCRTC・MFP・VM時間管理を分けて確認する

実測した約60 Hzを、信号生成の流れと対応付ける

1 CRTC

R04～R07の変更が垂直周期へ反映されるか確認する。

2 MFP

V-DISPイベントがGPIP bit 4へ届く処理を確認する。

3 VM時間

CPU実行と映像イベントを実時間へ合わせる処理を確認する。

実装の対応関係

```
CRTC register write
-> vertical timing update
-> V-DISP event
-> MFP GPIP update
-> VM time synchronization
```

Source(VisualStudio2019).zipから引用

出典表記：Source(VisualStudio2019).zipから引用

正しい実装は映像信号とホスト描画を分けている

必要な処理	XEiJ	XM6 TypeG	改善条件
CRTC値から周期を計算	ラスタ境界を再計算	CRTC処理で管理	固定fpsにしない
V-DISPを生成	MFPイベントを呼ぶ	MFPへ状態を反映	GPIP bit 4を独立生成
実時間へ同期	イベント時刻を累積	VM時間で進行	ホスト描画と分離
設定変更を反映	CRTC処理を再開	タイミングを更新	R04書込み後に再計算

描画フレームを省略しても、CRTCイベントは省略しない。

WebX68kではホスト画面更新とCRTC周期を切り分ける

```
X68000 60 Hz V-DISP measurement
```

```
Measuring 600 frames...  
Progress   : 540 / 600  
FPS : 57.14
```

```
ESC : abort
```

問題が生じる構造

```
requestAnimationFrame()  
  -> run one guest frame  
  -> update V-DISP  
  -> present canvas
```

WebX68k / PX68k系実装の調査

ホストの画面更新をX68000の1フレームとして扱うと、CRTC変更を表現できない

1

固定更新

R04を変更しても1回の更新時間が変わらない。

2

信号が描画依存

V-DISPがCanvas更新回数へ結び付く。

3

省略の影響

描画スキップで信号イベントまで失われる可能性がある。

WebX68kで観測 : FPS 57.14 (計測中の表示)

WebX68kはCRTCイベントをrequestAnimationFrameから独立させる

エミュレーション時間で信号を作り、ブラウザ更新は表示だけに使う。

1

周期を再計算

R04と水平周期からframe periodを求める。

2

GPIPをイベント化

R06/R07の位置でbit 4を変化させる。

3

描画を分離

遅れた場合は表示だけを省略する。

改善案（擬似コード）

```
while (emuTime < hostTargetTime) {  
    runCpuUntil(nextCrtcEvent);  
    updateRasterAndGpip();  
}  
  
if (frameImageReady) {  
    presentOnRequestAnimationFrame();  
}
```

改善案

55.5 Hzと60 Hzの両方を同じテストで測定できる状態を目標にする。

MPX68kでは固定55.45 HzステップがCRTC変更を隠す

```
X68000 60 Hz V-DISP measurement
```

```
Measuring 600 frames...  
Progress   : 540 / 600  
FPS : 57.14
```

```
ESC : abort
```

ホスト側スケジューラ

```
emulatorHz = 55.45  
targetFrameTime = 1.0 / 55.45
```

```
while accumulator >= targetFrameTime {  
    X68000_Update(clockMHz, 0)  
}
```

MPX68Kホスト側コード

画面を60 fpsで表示しても、ゲスト側が固定55.45 HzならV-DISPは60 Hzにならない。

1

固定周期

R04と無関係に更新間隔が決まる。

2

表示設定は別

ホストfpsを変えてもゲスト周期は残る。

3

CPU MHzも別

1更新の処理量だけでは垂直周期を直せない。

参考：[WebX68k](#)で観測された同種の症状。MPX68Kの実行画面ではない。

MPX68kはコアが計算したCRTC周期をホストへ渡す

固定周期を、現在のCRTC設定から求めた周期へ置き換える

1 コアで計算

水平周期 × 総ライン数からframe periodを返す。

2 ホストで同期

経過実時間だけコアを進める。

3 描画は結果

完成した画面だけSpriteKitへ渡す。

改善案（インターフェース）

```
typedef struct {
    uint64_t frame_period_ns;
    uint32_t total_lines;
    uint32_t vdisp_start;
    uint32_t vdisp_end;
} crtc_timing_t;

void crtc_get_timing(crtc_timing_t *t);
```

改善案

10/16 MHzの設定を変えても、同じCRTC設定なら垂直周期は変えない。

標準55.5 Hzと変更後60 Hzの両方で修正を確認する

確認項目	標準モード	525ライン設定
垂直周波数	約55.5 Hz	約60.0 Hz
600 V-DISP	約10.81秒	約10.00秒
GPIP bit 4	各フレームで変化	各フレームで変化
ホスト60/120 Hz	測定値は不変	測定値は不変
CPU 10/16 MHz	周期は不変	周期は不変

描画負荷を増やしてもV-DISP周期が変わらないことを確認する。

31 kHzモードで60 Hz垂直同期を生成・実測し、エミュレータの改善点を整理した

- 1 映像信号の4区間とR04～R07の役割から、525ラインの設定値を導出した。
- 2 MS.Xの実装を参考にC実験を作り、600回のV-DISPが約10秒になることを確認した。
- 3 複数エミュレータを比較し、CRTC由来の時間でV-DISPを生成する必要性を整理した。

謝辞

本資料を支えた公開資料、ソースコード、そして知識の共有に感謝する。

本資料の作成にあたり、MS.X、XEiJ、XM6 TypeG、PX68k、WebX68k、MPX68Kおよびelf2x68kの公開資料とソースコードを参照した。これらの成果を公開・維持している開発者の方々に深く感謝する。

また、長年にわたり技術資料、開発環境、ソフトウェアおよび検証結果を蓄積・共有してきたX68000コミュニティの皆様に謝意を表する。

X68000 COMMUNITY

参考文献

本文中の [n] は以下の文献番号に対応する。Web資料の参照日は2026年8月18日。

[1] kunichiko, 『MS.X開発秘話 — X68000で動かすMSX、30年越しのものづくり』, 電子版 v1.0, 2026.

<https://kunichiko.booth.pm/items/8672923>

[2] kunichiko, “X68000_MSX_Simulator,” GitHub repository, commit 55cc131, 2026.

https://github.com/kunichiko/X68000_MSX_Simulator

[3] stdkmd, “XEiJ,” CRTC.java and misc-include-vector.equ, source code.

<https://stdkmd.net/xeij/source/xeij-CRTC.java.htm>

[4] NXP Semiconductors, MC68901 Multi-Function Peripheral User’s Manual.

<https://www.nxp.com/docs/en/reference-manual/MC68901UM.pdf>

[5] XM6 TypeG project, “Source(VisualStudio2019).zip,” source archive.

<http://retropc.net/pi/xm6/index.html>

[6] hissorii et al., “PX68k / px68k-libretto,” GitHub repositories, commits 74ddbb3 / bb9b464.

<https://github.com/hissorii/px68k>

[7] uraraworks, “WebX68k,” main.ts and DESIGN.md, commit b0cd3c7.

<https://github.com/uraraworks/WebX68k/tree/b0cd3c7676d702be5d977a593d40fac825a1139c>

[8] YosAwed, “MPX68K,” GameScene.swift and PX68k core, commit 42fc340.

<https://github.com/YosAwed/MPX68K>